

 FORMATO PARA ELABORACIÓN DE TALLERES ESTUDIANTILES	Departamento: CIENCIAS DE LA COMPUTACION
	Carrera: Electrónica Y Automatización Asignatura: Fundamentos De Programación NRC: 28561
	Apellidos y nombres del estudiante: Angamarca Cuchiye Lenin Jhosue

TRIÁNGULO RECTÁNGULO HUECO DE 10 LÍNEAS

ANTECEDENTES

En la guía práctica se desarrolló un ejercicio utilizando estructuras repetitivas FOR y validación con Mientras para generar un triángulo rectángulo hueco mediante asteriscos.

El ejercicio permite comprender el uso de ciclos anidados, condiciones lógicas y validación de datos en PSeInt y lenguaje C.

Se utilizó la condición: $j = 1$ o $j = i$ o $i = n$ para determinar cuándo imprimir un asterisco y cuándo dejar espacios vacíos.

El desarrollo del ejercicio ayuda a fortalecer la lógica de programación y la traducción de algoritmos desde pseudocódigo hacia Code Blocks.

OBJETIVO

Aplicar estructuras repetitivas FOR y validaciones con repetir en la construcción de algoritmos.

Diseñar un programa capaz de generar un triángulo rectángulo hueco utilizando caracteres.

Traducir correctamente un pseudocódigo desarrollado en PSeInt hacia lenguaje C y analizar el funcionamiento del algoritmo mediante una prueba de escritorio.

DESARROLLO

Antes de programar en C, conviene escribir la solución en PSeInt. Aquí se observa la lógica con palabras cercanas al lenguaje natural.

Realizar un triángulo hueco utilizando (*) de 10 líneas.

SEUDOCÓDIGO PARA PSEINT

Algoritmo TrianguloHueco

Definir i, j, n Como Entero

Repetir

 Escribir "Lineas del triangulo: "

 Leer n

 Si $n \leq 0$ Entonces

 Escribir "Ingrese un numero mayor que 0: "

 FinSi

Hasta Que $n > 0$

Para i <- 1 Hasta n Con Paso 1 Hacer

 Para j <- 1 Hasta i Con Paso 1 Hacer

 Si $j = 1$ O $j = i$ O $i = n$ Entonces

 Escribir Sin Saltar "*" "

 SiNo

 Escribir Sin Saltar " " "

 FinSi

FinPara

Escribir ""

FinPara

FinAlgoritmo

PRUEBA DE ESCRITORIO EN PSEINT

La prueba de escritorio permite comprobar manualmente qué hace el algoritmo. Para que sea fácil de revisar, se muestra el comportamiento cuando $n = 10$.

Fila i	Rango de j	Condición que dibuja borde	Salida de la fila	Explicación breve
1	1	$j=1$ y $j=i$	*	Primera fila: solo un asterisco.
2	1 a 2	$j=1$ o $j=i$	* *	Tiene borde izquierdo y diagonal.
3	1 a 3	$j=1$ o $j=i$	* * *	El centro queda vacío.
4	1 a 4	$j=1$ o $j=i$	* * *	Aumenta el espacio interno.
5	1 a 5	$j=1$ o $j=i$	* * *	Se mantienen los bordes.
6	1 a 6	$j=1$ o $j=i$	* * *	Continúa el patrón hueco.
7	1 a 7	$j=1$ o $j=i$	* * *	Más columnas internas vacías.
8	1 a 8	$j=1$ o $j=i$	* * *	El borde diagonal se desplaza.

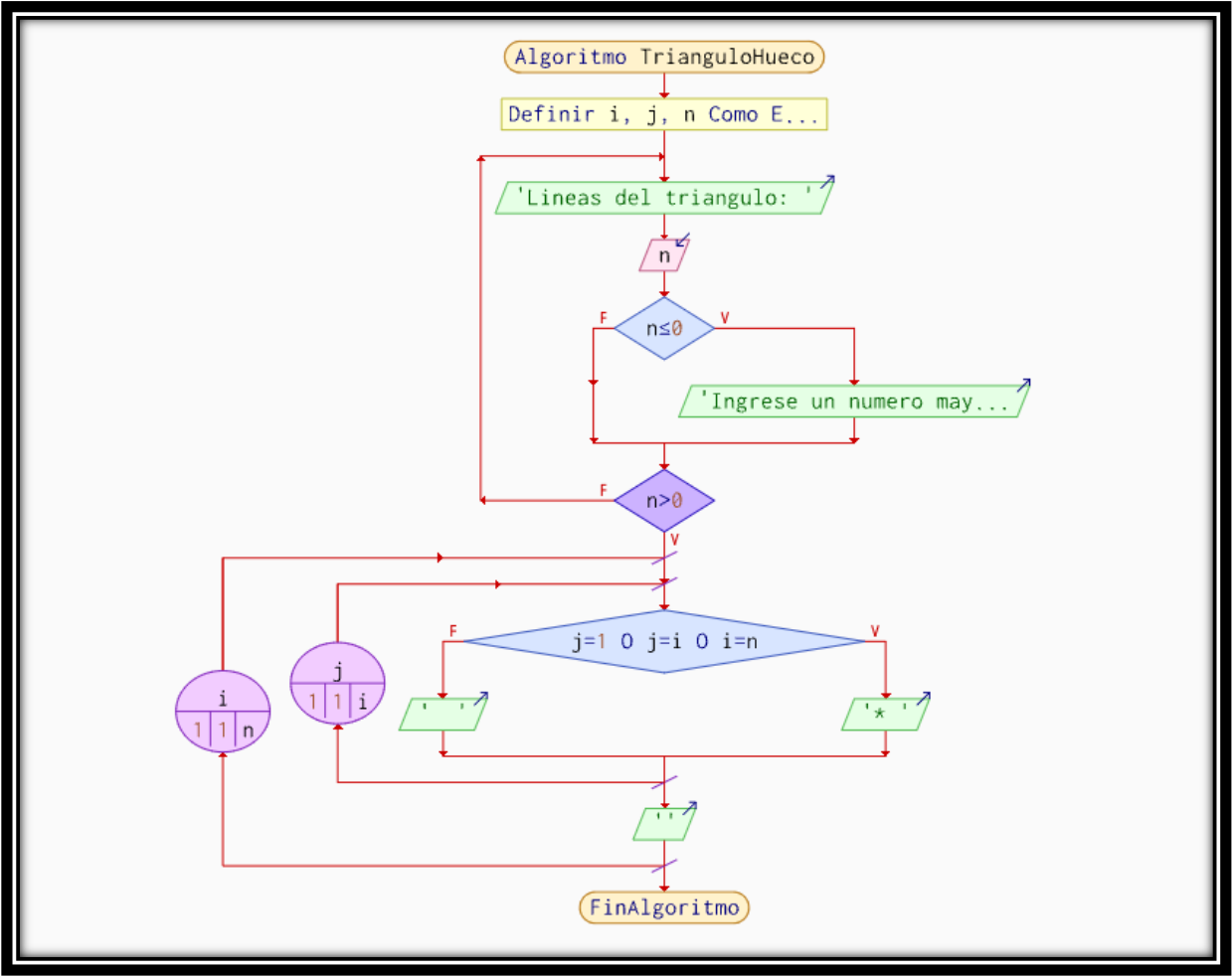
9	1 a 9	$j=1$ o $j=i$	* *	Última fila antes de la base.
10	1 a 10	$i=n$	* * * * * * * * * * * * * * *	Base completa del triángulo.

```

PSeInt - Ejecutando proceso TRIANGULOHUECO
*** Ejecución Iniciada. ***
Lineas del triangulo:
> -2
Ingrese un numero mayor que 0:
Lineas del triangulo:
> 10
*
* *
*  *
*   *
*    *
*     *
*      *
*       *
*        *
* * * * *
*** Ejecución Finalizada. ***
☐ No cerrar esta ventana ☐ Siempre visible Reiniciar

```

DIAGRAMA DE FLUJO



CODE BLOCKS

Ahora sí: pasamos la lógica a lenguaje C. La clave es traducir cada instrucción de PSeInt a su equivalente en C, sin perder el orden de los ciclos.

PSEINT	C / CODE::BLOCKS	PARA RECORDAR
Algoritmo ... FinAlgoritmo	int main() { ... return 0; }	Todo programa en C inicia en main.
Definir i, j, n Como Entero	int i, j, n;	Las variables enteras se declaran con int.

Escribir	printf();	printf muestra texto o resultados.
Leer n	scanf("%d", &n);	El & indica la dirección donde se guarda el dato.
Repetir ... Hasta Que n > 0	do { ... } while (n <= 0);	El bloque se ejecuta al menos una vez y se repite mientras la condición sea verdadera.
Si n <= 0 Entonces	if (n <= 0)	Permite validar si el número ingresado es incorrecto.
Escribir "Ingrese un numero mayor que 0:"	printf("Ingrese un numero mayor que 0:\\n");	\\n realiza un salto de línea.
Para i <- 1 Hasta n Con Paso 1 Hacer	for (i = 1; i <= n; i++)	El primer FOR controla las filas.
Para j <- 1 Hasta i Con Paso 1 Hacer	for (j = 1; j <= i; j++)	El segundo FOR controla las columnas.
Si ... Entonces	if (...)	La condición usa operadores lógicos.
O		En C, OR se escribe con doble barra vertical.
= para comparar	==	En C, == compara; = asigna.
Escribir Sin Saltar	printf("...");	No se coloca \\n para continuar en la misma línea.
Escribir ""	printf("\\n");	Permite bajar a la siguiente fila del triángulo.

Código validado en C para Code::Blocks

```
#include <stdio.h>
```

```

int main()
{
    int i, j, n;

    do
    {
        printf("Lineas del triangulo: ");
        scanf("%d", &n);

        if (n <= 0)
        {
            printf("Ingrese un numero mayor que 0:\n");
        }

    } while (n <= 0);

    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= i; j++)
        {
            if (j == 1 || j == i || i == n)
            {
                printf("* ");
            }
            else
            {

```

```
"C:\Users\Admin\Documents\ANGAMARCA L.A\CODE BLOCKS\TITLE\Untitled123.exe"
Lineas del triangulo: -5
Ingrese un numero mayor que 0:
Lineas del triangulo: 10
*
* *
*  *
*   *
*    *
*     *
*      *
*       *
*        *
* * * * *
* * * * *

Process returned 0 (0x0)   execution time : 9.657 s
Press any key to continue.
```

Las estructuras repetitivas FOR permiten controlar filas y columnas para generar figuras mediante caracteres.

Código de documento: FOR-TALL-202650-001 Rev.: 01 Código de proceso: DOC-EST-01

La prueba de escritorio contribuye a comprobar el funcionamiento del algoritmo antes de programarlo, paso a paso.

RECOMENDACIONES

Para prevenir errores, siempre valide la información que el usuario ingresa.

Realizar ensayos con diversos números de líneas para verificar cómo funciona el algoritmo.

ANEXOS (opcionales/RUBRICA DE LA GUIA)

Guia_FP_Ejercicio_FORV1.0_Mientras.docx

RUBRICA DE EVALUACION

Criterio	Excelente	Bueno	En proceso	Puntaje
Comprensión del problema	Identifica claramente que el triángulo debe ser hueco, con borde izquierdo, diagonal y base.	Reconoce la forma del triángulo, aunque explica parcialmente la condición de impresión.	Confunde triángulo lleno con triángulo hueco o no distingue los bordes.	4 pts
Seudocódigo en PSeInt	Usa correctamente variables, Repetir, Para, Si/Sino y Escribir Sin Saltar.	Presenta la estructura general, con pequeños errores de sintaxis o ubicación.	El pseudocódigo está incompleto o no representa la lógica del ejercicio.	4 pts
Prueba de escritorio	Comprueba el algoritmo fila por fila y explica la salida para $n = 10$.	Realiza una prueba parcial con algunas filas o poca explicación.	No evidencia seguimiento de variables i y j o la salida no coincide.	4 pts
Traducción a C en Code::Blocks	El código compila, ejecuta y mantiene la lógica validada del triángulo hueco.	El código tiene errores menores corregibles, pero la lógica principal es adecuada.	El código no compila o no aplica correctamente FOR, if y operadores lógicos.	5 pts
Presentación y orden	Entrega el trabajo limpio, ordenado y fácil de leer, con comentarios útiles.	El trabajo es comprensible, aunque puede mejorar en orden o comentarios.	El trabajo es difícil de seguir o no evidencia organización.	3 pts